A PROJECT REPORT ON

Project MAVIS: Media, Authenticity, Verification and Integrity System

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Chinmay Patil	Exam No: B400050532
Omkar Wagholikar	Exam No: B400050637
Shantanu Wable	Exam No: B400050634
Soaham Pimparkar	Exam No: B400050551

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025

CERTIFICATE

This is to certify that the project report entitled

PROJECT MAVIS
(MEDIA AUTHENTICITY VERIFICATION AND INTEGRITY SYSTEM)

Submitted by

CHINMAY PATIL	Exam No: B400050532
OMKAR WAGHOLIKAR	Exam No: B400050637
SHANTANU WABLE	Exam No: B400050634
SOAHAM PIMPARKAR	Exam No: B400050551

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. M. S. Wakode** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. M. S. Wakode
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date: 25/04/2025

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***'Project MAVIS - Media Authenticity Verification and Integrity System'****.*

We would like to thank our project guide ***Prof. M. S. Wakode*** *for giving us all the help and guidance we needed. We are really grateful to her for her kind support and valuable suggestions.*

We are also grateful to ***Dr G. V. Kale****, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to* ***Dr. A. R. Deshpande****, BE project coordinator, for her timely guidance and support.*

In the end our special thanks to our mentor ***Mr. Tushar Sugandhi****,* ***StellarBlocks LLC****. for his continued support and sponsorship of the problem statement. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Chinmay Patil
Omkar Wagholikar
Shantanu Wable
Soaham Pimparkar
(B.E. Computer Engg.)

ABSTRACT

As generative models create increasingly convincing text, audio, images, and video, the potential for misinformation and disinformation has grown, undermining public trust in digital media. Traditional methods of detecting deepfakes often fall short due to the complexity and evolving nature of generative models. Therefore, this project and its preliminary report explores an alternative solution: Digital integrity system for embedding and verifying media authenticity information. Rather than identifying a deepfake piece of media, this project proposes a system to establish credibility or verify a genuine piece of media using tamper-evident hashes, signatures enabling non-repudiation, capture and verification mechanisms. Leveraging cryptographic techniques, Trusted Execution Environments (TEE) or Secure Enclaves, and secure metadata tagging, we aim to establish the proof of non tamper. By focusing on securing the source, rather than solely relying on detection algorithms, we offer a proactive solution to combat the growing threat of deepfakes. The project's objectives are to design and implement a media authenticity and integrity verification framework that is resilient to tampering and provides high confidence in the authenticity of media for end-users.

Contents

List of Figures

CHAPTER 1
INTRODUCTION

1.1 Motivation

The rapid development of generative models has made media manipulation not only more accessible but also more difficult to detect, with deepfakes and synthetic content increasingly blending into our digital environment. Traditional detection algorithms have proven inadequate due to the adaptable nature of these techniques, which continuously evolve to bypass standard safeguards. This challenge motivates a shift from mere detection to provenance-based verification, where tamper-resistant authenticity is established at the source, empowering users to verify content integrity independently. This approach is particularly relevant in fields where authenticity is critical—such as journalism, legal evidence, insurance claims and surveillance—where trust in media has profound implications.

1.2 Problem Definition

The advent of advanced generative models has introduced a new wave of highly convincing fake media, including deepfakes and synthetic content, which have significantly increased the risk of misinformation. Traditional detection methods struggle to keep up with the sophistication and adaptability of these models, often failing to identify manipulated content reliably. This gap necessitates a solution that ensures media authenticity at the point of capture, creating a foundation for content verification that is secure, tamper-evident, and resistant to subsequent manipulation. Existing solutions do not provide an adequate framework for establishing and verifying content provenance, especially in environments where media integrity is crucial, such as journalism, legal documentation, and digital forensics.

1.3 Objectives

1. **Design a Robust Media Authenticity Framework**: Develop a comprehensive system to capture, timestamp, and secure media on user devices using Trusted Execution Environments (TEEs) or Secure Enclaves for enhanced security.

2. **Implement Cryptographic Provenance Verification**: Incorporate digital signatures, tamper-evident hashes, and timestamps from a Trusted Timestamping Authority to ensure media is non-repudiable and traceable to its source.

3. **Embed Invisible Watermarking Techniques**: Use GAN-based or autoencoder watermarks to create invisible, tamper-resistant marks within the media, ensuring authenticity even after compression or other transformations.

4. **Enable Server-Side Verification and Re-Signing**: Design a server protocol that recalculates the hash, verifies metadata, and re-signs media for additional credibility, enhancing trust across the lifecycle of the media.

5. **Develop an End-User Verification Platform**: Create a platform that allows consumers to independently verify media provenance, authenticity, and integrity, ensuring transparency and trust in digital content.

1.4 Methodology of Problem Solving

This project was developed through a structured research and design approach, beginning with an extensive literature review to identify key challenges and existing solutions in the area of media authenticity and provenance verification. We surveyed industry standards, research papers, patents, and existing products, particularly focusing on cryptographic techniques, trusted execution environments (TEEs), GAN-based watermarking, and blockchain applications in media integrity.

The methodology for implementing the proposed system for embedding and verifying media authenticity information is divided into five main phases:

1. **Capture**: Securely capture media with cryptographic hash and metadata embedding in a Trusted Execution Environment (TEE).

2. **Watermarking**: Embed an invisible GAN-based watermark for tamper resilience.

3. **Server Verification**: Recalculate hash, verify TEE signature, and re-sign for added chain-of-custody security.

4. **Encryption and Transmission**: Encrypt metadata and watermark, transmitting data securely to the server.

5. **End-User Verification**: Allow users to verify metadata, signature, and watermark to confirm media integrity and provenance.

CHAPTER 2
LITERATURE SURVEY

In the research phase, we conducted a comprehensive survey of existing literature, patents, and industry standards to understand current advancements and limitations in media authenticity and provenance verification. This included analyzing research papers on cryptographic watermarking, trusted execution environments (TEEs), GAN-based watermarking techniques, and deepfake detection. We also examined industry standards, such as those from the Coalition for Content Provenance and Authenticity (C2PA), and existing solutions in the market. This groundwork provided a clear view of the technological landscape, helping us identify gaps and informing our architectural design.

Recent research has increasingly focused on content provenance and cryptographic solutions as a proactive alternative to combat the spread of manipulated media. This literary survey explores key studies and technological advancements in the field of tamper-resistant media systems, examining the role of various techniques in building frameworks for video authenticity.

In a 2023 study, Feng et al. [1] conducted an online experiment involving 595 participants from the US and UK to examine the impact of provenance information on users' trust and perceptions of accuracy regarding visual content shared on social media. The findings indicated that when users were presented with provenance details, particularly in cases where the content was identified as composited, trust was generally reduced, and users were more skeptical of potentially misleading media. The researchers also looked at scenarios where provenance information was incomplete or incorrect. In these cases, participants rated the media as less accurate and trustworthy compared to content that did not exhibit these issues.

Bhowmik et al. (2024) [2] present a paper discusses the development of an international standard for assessing the trustworthiness of media, particularly in a digital landscape where disinformation is rampant. The standard aims to provide a universal framework for evaluating media credibility, focusing on the role of provenance, metadata, and integrity verification techniques to ensure reliable media content across borders.

C2PA [3] is an open standard designed to verify the authenticity and integrity of digital content by embedding tamper-evident metadata and cryptographic signatures. Developed by a coalition that includes Adobe, Microsoft, and Truepic, it enables users to trace the provenance of media files and ensures that the content has not been altered or tampered with after creation.

Black et al. (2021) [4] propose a self-supervised network to develop a robust video fingerprinting model by encoding both visual and audio streams from videos. Using contrastive learning and a variety of data augmentations, the model is trained to handle diverse inputs. For partial video matching, the approach divides videos into fixed-length chunks (10 seconds each), created through a sliding window mechanism with overlap, ensuring that each chunk is indexed and searchable independently. This chunk-level indexing uses an inverted index, similar to text search systems, where video chunks are represented as "bags of features" and mapped to codewords using a KMeans-based dictionary.

Andriushchenko et al. (2022) [5] propose ARIA (Adversarially Robust Image Attribution), a method designed to improve the robustness of image attribution systems against adversarial attacks, which involve imperceptible manipulations intended to deceive deep learning models. Image attribution, which matches an image to a trusted source to verify authenticity, is crucial in fighting online misinformation. However, current models are vulnerable to adversarial perturbations, causing incorrect attributions or failure in detecting manipulations. ARIA employs robust contrastive learning to strengthen the defense of image attribution models, ensuring resilience against adversarial examples while maintaining high accuracy on unaltered images. The proposed method is computationally efficient and significantly outperforms prior approaches, achieving up to 85.1% recall under adversarial attacks compared to 0.0% in earlier models. The authors also demonstrate that the approach generalizes well to various types of unseen adversarial perturbations.

Zhang et al. (2020) [6] explores the different ways and challenges to protecting the IP of images generated by Deep Neural Networks. The paper proposes securing the ip of production level deep learning models by an watermark implanting approach secure model ownership. The core idea behind the framework being to train the model on a specifically crafted watermarked dataset with pre determined predictions which can then be used to remotely verify the model ownership. It also highlights the increased targeting of DNN models leading to copyright infringement as well as shedding light on drawbacks of some previous watermarking attempts.

Van Le and Desmedt (2015) [7] conducted cryptanalysis of different watermarking schemes designed for intellectual property protection by the UCLA team. The core concept being to embed a signature into the digital property to assert one's ownership to the property. The paper studies watermarking methods based on the graph coloring problem and different schemes

including the FPGA circuits for embedding signatures. The paper also outlines the necessary requirements for the effective watermarking. While also shedding light on the vulnerabilites of some of the schemes proposed by the UCLA team as well as highlighting the implicit limitations of watermarks that solely rely on their secrecy of location.

Petrangeli et. al. (2024) [8] present a client-side implementation of C2PA validation for DASH (Dynamic Adaptive Streaming over HTTP) video streaming, enabling real-time verification of video authenticity while ensuring minimal impact on performance. The integration of content provenance mechanisms during streaming provides a seamless approach to verifying the integrity of video content as it is delivered to end-users. This work is a crucial step toward scalable, tamper-resistant video streaming solutions, particularly in contexts where media manipulation is prevalent.

Sedlmeir et. al. (2023) [9] present a paper that examines how users perceive provenance-enabled media and its influence on trust and accuracy. Through user experiments, the research reveals that embedding provenance information within digital media enhances users' confidence in the authenticity of the content, reducing susceptibility to misinformation. These findings highlight the potential of provenance technology in improving public trust in digital media ecosystems, which is essential for combating disinformation.

Nikolaos et. al. (2024) [10] investigates how to balance the privacy of users with the need for provenance data in images. It proposes methods for privacy-preserving metadata that allow users to verify the authenticity of an image without exposing sensitive personal information, such as location data. This research is crucial for implementing provenance systems that respect privacy while ensuring media integrity and authenticity.

Bureacă et. al. (2024) [11] conducted a research that proposes a blockchain-based framework for ensuring the authenticity and provenance of digital content. By storing immutable records of media origins on a distributed ledger, the system offers a decentralized approach to verifying content authenticity. This method ensures tamper-proof tracking of media, making it highly resilient to manipulation and providing an auditable trail for content verification.

CHAPTER 3
PROJECT PLAN

3.1 Project Estimate

- **Development Timeline**: 8 months (September 2024 - April 2025)
- **Resources Required**:
  1. **Talent / Knowledge Requirements**:
     - Cryptography Engineer
     - Full Stack Developer
     - Multimedia Engineer
     - App Developer
     - UI/UX Designer
     - Scrum Master
     - QA/Test Engineer
     - Deep Learning Engineer
     - Technical Writer
  2. **Software, Tools and Infrastructure**:
     - Development tools (IDEs, version control)
     - Cloud Hosting (Azure) for the development, test, and production environments.
     - C2PA SDK/API integration for media provenance
     - Media processing libraries for handling JPEG, MP4, and other file formats
     - Cryptographic libraries for signing and verification
     - Metadata management and tagging tools
  3. **Paid resources/ Expenses**:
     - Software Tools & licenses
     - Publication and Patenting costs (if required)
     - Hosting and Deployment
     - Contingency Costs

3.2 Risk Management

In this section, we outline the process of identifying potential risks that could impact the project's success, analyzing their likelihood and potential impact, and developing strategies to mitigate, monitor, and manage these risks. This proactive approach ensures that challenges are addressed effectively to minimize disruptions during the project's implementation and operation phases.

3.2.1 Risk Identification

- **Technical Risk**:
    1. Challenges in embedding tamper-resistant features at the point of media creation across different devices.
    2. Ensuring robust cryptographic security in environments where metadata could be stripped during sharing
    3. Handling media compression without invalidating cryptographic signatures.
    4. Media file format conversion without losing the signatures and other provenance information.
    5. Invisible watermarking using GANs or autoencoders may be susceptible to distortion or loss during compression, reformatting, or intentional tampering.
- **Operational Risk**:
    1. Difficulty finding experts skilled in the domain of cryptographic media provenance.
    2. Delays in adopting hardware identifiers or secure signing mechanisms across different device ecosystems.
    3. Risk of propagation of sensitive information through provenance data within the metadata
- **Market Risk**:
    1. Competitors like Truepic and other provenance verification systems gaining wider adoption.
    2. End-user reluctance to adopt the verification framework, especially if understanding of media provenance technology is low.

3.2.2 Risk Analysis

- **High Impact/High Likelihood**: Issues related to hardware integration for signing and media provenance at the source, as the hardware ecosystem is fragmented.
- **Medium Impact/Medium Likelihood**: Difficulty in maintaining metadata integrity when media passes through platforms that strip or alter metadata.
- **High Impact/Low Likelihood**: Potential method to bypass the system integrity and change or read the encrypted information.

3.2.3 Overview of Risk Mitigation, Monitoring, Management

1. Mitigation Plan
    - Communication with industry experts to collaborate on integrating secure signing mechanisms at the device level.
    - Exploring various technologies and solutions (e.g. Central database, Blockchain, etc) to store metadata and provenance information that remains verifiable even if stripped.
    - Implement performance and compatibility testing for metadata robustness during file format changes.
2. Monitoring
    - Continuous monitoring of project milestones through project management tools.
    - Automated performance testing to ensure media signature verification remains intact after transformations like compression or social media sharing.
3. Management
    - Weekly project reviews scheduled with the external guide to report progress, discuss the queries and decide the next steps
    - Microsoft suite and project management tools for internal communication and collaboration within the team.

3.3 Project Schedule

3.3.1 Project Task Set

- **Phase 1: Initial Planning & Design** (September 2024).
  - Project team, guide and industry sponsorship (Stellarblocks LLC) is finalized.
  - Team and guide communication, exploring and analyzing various project ideas.
  - Idea finalization, primary research and synopsis writing.
- **Phase 2: Research and Survey** (October 2024)
  - Survey of relevant papers, patents, products and services.
  - Understanding the stakeholders, market needs and gaps to find a PoI (Point of Intervention).
  - Communication with relevant professionals, companies and other stakeholders.
  - Drafting the system design and architecture, finalizing team roles, defining project timeline and goals.
- **Phase 3: Prototyping and Revision** (November-December 2024)
  - Learning the necessary skills and technologies.
  - Building the prototype(s), recognizing and solving the obstacles.
  - Proposing a revised architecture based on new insights and finalizing the system design.
- **Phase 4: Implementation** (January- February 2025)
  - Implementation of the final design.
  - Drafting a research paper.
- **Phase 5: Launch and Publishing** (March- April 2025)
  - Launching the solution through the appropriate medium (as an online service/ open source product/ other)
  - Publishing the research paper.
  - IP management, documentation and Handover.

3.3.2 Timeline Chart

Figure 5.1 is the summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

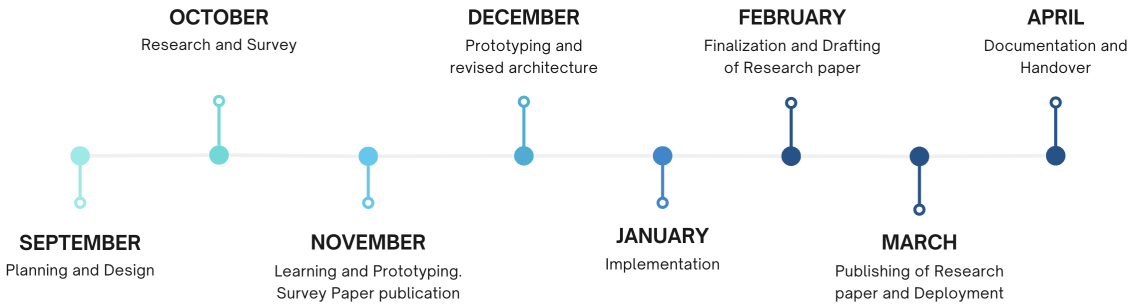

Figure 3.1: Summarized Timeline Chart

3.4 Team Organization

3.4.1 Team structure

- **Chinmay Patil:** iOS Developer, System Designer.
- **Omkar Wagholikar:** Cybersecurity Engineer, System Designer, Backend Developer.
- **Shantanu Wable:** UI/UX Design, Backend Developer.
- **Soaham Pimparkar:** Deep Learning Engineer, Cryptography.
- **Prof. Madhuri Wakode:** Internal mentor and guide.
- **Tushar Sugandhi (Stellarblocks LLC):** Project manager, sponsor and external guide.
- **StellarBlocks LLC:** Project Sponsor and Industry connect.

3.4.2 Management reporting and communication

- **Weekly Reporting**: Weekly online meetings have been scheduled with the external guide for discussing the queries, further steps and reporting the progress.
- **Collaboration and Tools**: Microsoft suite is used for internal communication and collaboration allowing the sharing and discussions outside the decided meeting times.

CHAPTER 4

SOFTWARE REQUIREMENTS SPECIFICATION

4.1 Introduction

4.1.1 Project Scope

The scope of this project is centered around developing a system for embedding and verifying media authenticity information. It allows users to capture and securely sign photos and videos at the point of creation using timestamp and device's TPMs (Trusted Platform Modules) or TEEs (Trusted Execution Environment) such as Apple's Secure Enclave. This signature will be verified by a server to ensure media authenticity. The project focuses on visual media (photos and videos) and does not include audio or other types of digital content. The primary objective is to enable users to reliably trace the source of media, ensuring a credible proof of provenance, even after the media is shared or modified outside the app. By embedding tamper-evident features and cryptographic signatures, the platform provides an essential layer of security for media authenticity and traceability.

4.1.2 Assumptions

In our project, several key assumptions guide the development of this system.

1. We have access to unique hardware identifiers, such as camera module identifiers, TPMs (Trusted Platform Modules) or TEEs (Trusted Execution Environment) such as Apple's Secure Enclave, or Secure Enclaves, on recording devices.

2. We assume that the media being shared in a loss-less manner throughout its lifecycle.

3. Critical sections of the metadata remain uneditable, aside from authorized manipulations made by our system.

4. We also presume that our system can efficiently handle the expected load and that any changes made to the media are non-editorial in nature.

5. Finally, we assume that the device used for capturing and proving media has access to both the internet and location services.

4.1.3 Use Cases

The potential use-cases for this project are vast, addressing key challenges in media authenticity and trust.

- Combating misinformation by ensuring the authenticity of news videos and preventing the spread of manipulated media across **social platforms**.
- Recording credible video and image proofs for insurance or warranty claims.
- Making surveillance footage tamper-proof and more credible as an evidence.
- Saving and sharing document soft copies in a tamper-resilient manner.
- **Content creators** and **digital marketers** can use the system to authenticate original media and protect intellectual property from tampering or unauthorized reuse.
- The system can also be applied in **journalism**, where verifiable media provenance is critical for maintaining trust in reporting.

4.2 Functional Requirements

4.2.1 Media Capture Requirements

The Media Capture Requirements for our system focus on ensuring secure and verifiable video recording at the point of creation.

1. **Camera Module**: The camera module must provide a unique identifier (UUID), which will be used in conjunction with the video for cryptographic signing, ensuring that each video can be reliably traced back to its source.
2. **Device-specific Hardware Identifiers**: Hardware-based security features like TPM, TEE or Secure Enclave to access the device's private key, enabling secure digital signatures on the media.

4.2.2 Data Processing Requirements

The Data Processing Requirements for the system are focused on ensuring secure and efficient handling of media throughout its lifecycle.

- **Media Streaming**: The system must be able to stream media to a remote server, allowing videos to be securely transmitted for further processing.

- **Media Signing**: The system needs to have the capability to sign media and compute cryptographic hashes, ensuring the integrity and authenticity of the content.
- **Storage**: Cloud storage is essential for storing videos in various formats and resolutions, along with their corresponding computed hashes for verification purposes.
- **IAM Keystore**: The system must securely manage private keys and handle personally identifiable information (PII), ensuring that sensitive user data and cryptographic materials are protected from unauthorized access

4.2.3 Media Verification Requirements

The Media Verification Requirements focus on ensuring the authenticity and integrity of the captured media through cryptographic methods. Specifically, the system must be able to compute cryptographic hashes of the media and verify digital signatures embedded during the media capture process. By computing hashes, the system generates a unique fingerprint of the video, which can then be compared against the original hash stored in the database to detect any tampering or alterations. The verification of digital signatures ensures that the media is from a legitimate source, using the device's private key embedded during recording. This process forms the backbone of the tamper-resilient system, ensuring that the video remains trustworthy and unmodified throughout its life cycle.

4.3 External Interface Requirements

4.3.1 User Interfaces

The proposed system is intended to feature a user-friendly iOS app.. This interface facilitates various essential functionalities, including enabling users to capture media directly from the camera, sign it, share it, and check the source of other media shared on the platform.

4.3.2 Hardware Interfaces

The system requires access to specific hardware devices to facilitate its operations effectively. This includes a mobile device which can be used to capture media. Being able to access the universally unique hardware identifier of the camera is necessary to track the source of the captured media. Geo location of the device can also aid in tagging the media. Storage options such as HDDs or SSDs are needed for data management. Secure Enclave for signing the media.

4.3.3 Software Interfaces

1. Operating System: Apple iOS 17 and above.
2. Azure VMs for storing videos and hashes, hosting server computing different formats of the media

4.4 Nonfunctional Requirements

4.4.1 Performance Requirements

The computation requirement is mainly focused around dividing the media into chunks, calculating and verifying their hashes to verify the authenticity of the media. That can be an expensive process on both the server as well as client side, but is essential.

1. **RAM Size:** Minimum 4gb on the client side.
2. **RAM:** Minimum LPDDR4
3. **CPU:** Apple A14 Bionic and above.

4. **Internet Speeds:** Minimum 10mbps Data downloads to and fro Google drives involves transferring data size in the neighborhoods of 1-2 gigabytes.

4.4.2 Safety Requirements

The safety requirements of this project focus on ensuring the secure handling of media, user data, and cryptographic materials. First, all private keys used for signing media must be securely stored, leveraging hardware security modules like TPM or Secure Enclave to prevent unauthorized access.

4.4.3 Security Requirements

The security requirements of this project are centered around protecting the integrity, authenticity, and confidentiality of media and associated data.

- **Strong Encryption**: All media files and their metadata must be safely encrypted during transmission and storage to prevent unauthorized access.
- **Access Control**: Robust access control and auditing must be implemented to monitor system interactions and prevent unauthorized use or data breaches.
- **Data Compliance**: Compliance with data protection regulations, such as GDPR, is critical to ensure the safe handling and storage of Personally Identifiable Information (PII), maintaining user privacy and security.

4.5 System Requirements

4.5.1 Database Requirements

The database requirements for this project focus on securely storing media files, metadata, and cryptographic signatures.

- **Relational Database**: PostgreSQL or MySQL (suitable for managing structured data such as metadata, user information, and cryptographic hashes)

- **NoSQL Database**: Apache Cassandra or MongoDB (handling unstructured or semi-structured data, such as different media formats and resolutions)
- **Scalability**: The database must support high availability and scalability to manage large volumes of media files, and it should offer encryption at rest to protect sensitive data. Efficient indexing and backup mechanisms are also required to ensure quick retrieval and data integrity in case of failures.

4.5.2 Software Requirements

The software requirements for this project, with an initial focus on iOS, include using development tools and frameworks that ensure security and performance.

1. The media app will be developed using **Swift**, leveraging Xcode as the primary development environment.
2. Apple's **CryptoKit** or **CommonCrypto** will be used to implement secure hashing and encryption.
3. Integration with **iOS Secure Enclave** to manage private keys securely.
4. For UI/UX, UIKit or **SwiftUI** will be employed to create an intuitive interface
5. Testing tools like **XCTest** will be used for ensuring the app's stability and performance under various conditions.

4.5.3 Hardware Requirements

The hardware requirements for this project, particularly on iOS devices, involve using components that ensure secure media capture and cryptographic operations.

- **Secure Enclave**: The device must support Secure Enclave, available in modern iPhones and iPads, to securely store and manage cryptographic keys.
- **Camera Module Hardware Identifier**: The camera module must provide a hardware identifier to be used in media signing.
- **Geolocation Access**

- **Internet Connectivity**
- **Sufficient Processing Power**: to handle real-time media signing, hashing, and secure data storage without performance bottlenecks

CHAPTER 5
SYSTEM DESIGN

5.1 System Design Diagram

The following diagram illustrates the overall system design for the application.

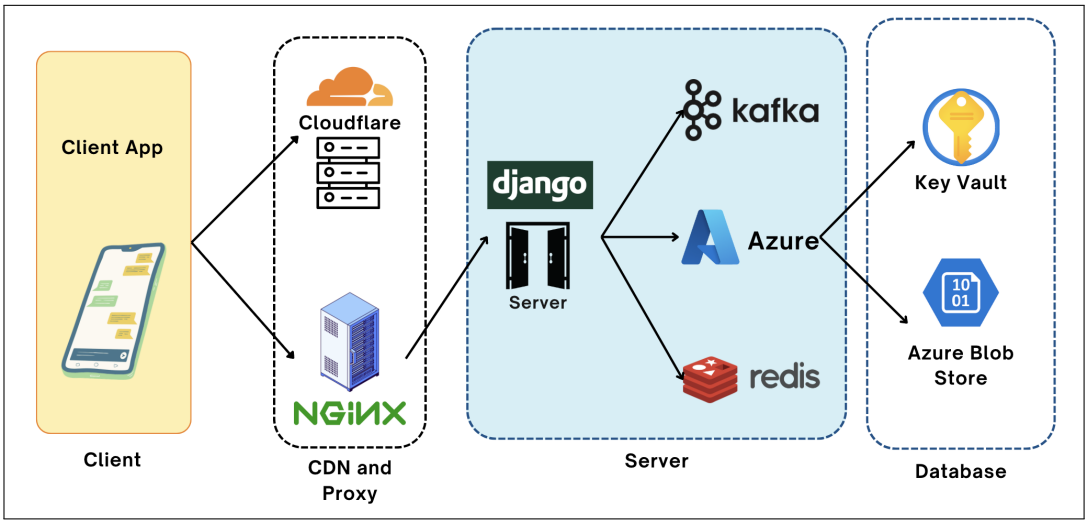

Figure 5.1: System Design Diagram

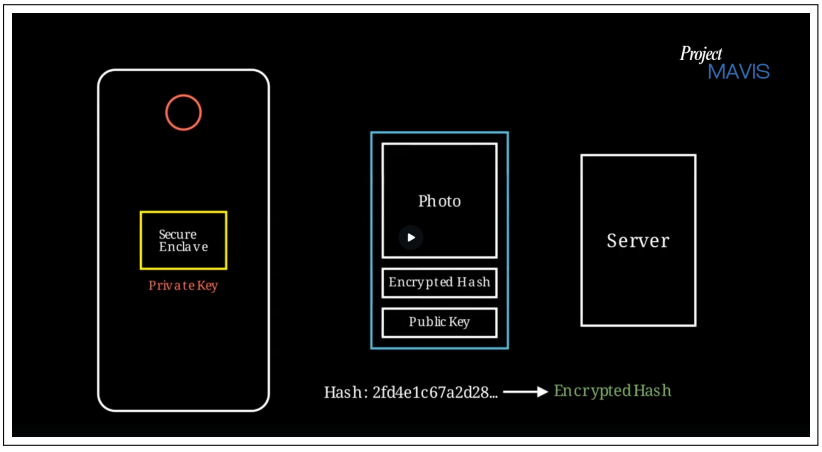

Figure 5.2: Step 1

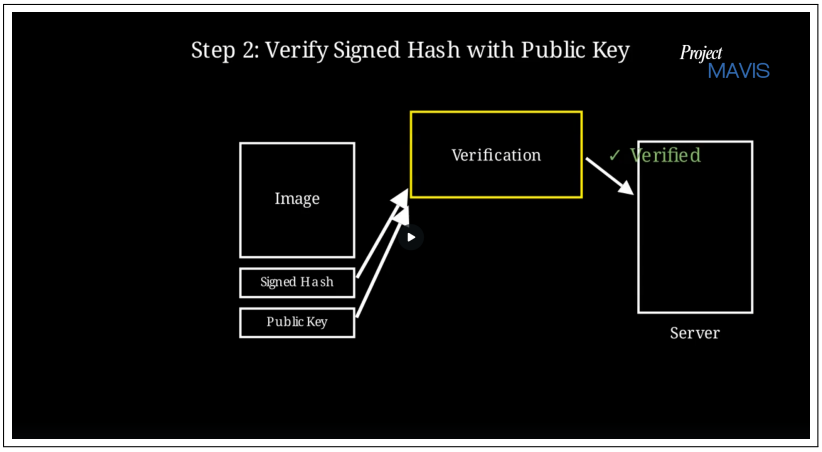

Figure 5.3: Step 2

Figure 5.4: Step 3

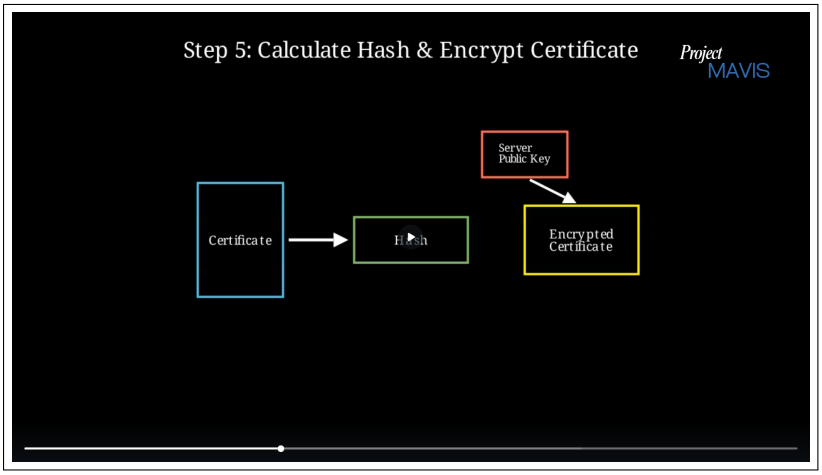

Figure 5.5: Step 5

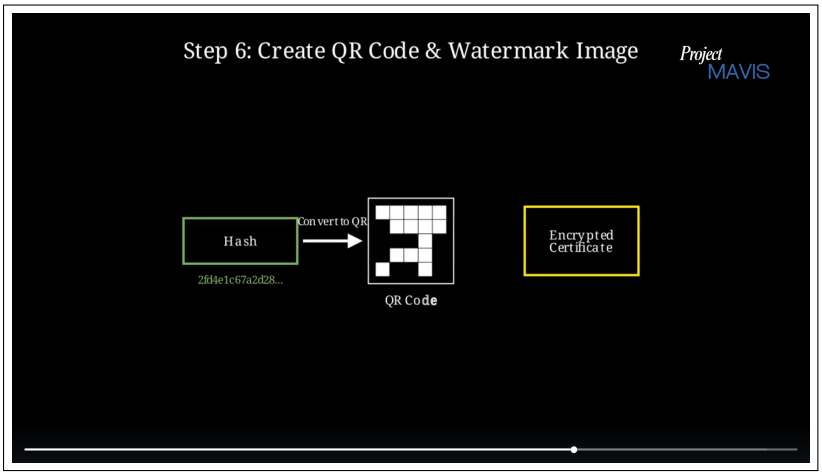

Figure 5.6: Step 6

5.2 System Architecture and UML Diagram

The following diagram illustrates the architecture of the system. It depicts how content producers and consumers will interact using the system architecture for their own needs.

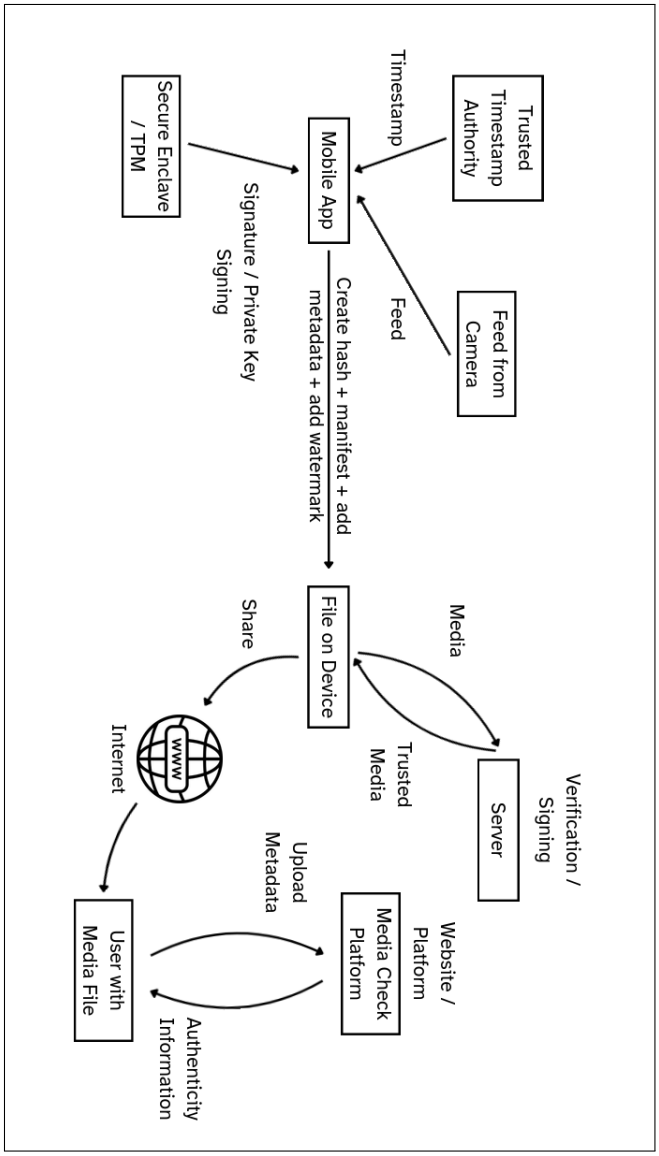

Figure 5.7: Architecture Diagram

5.3 Data Flow Diagram

The following diagram illustrates the flow of data between inputs, hash computations, and outputs.

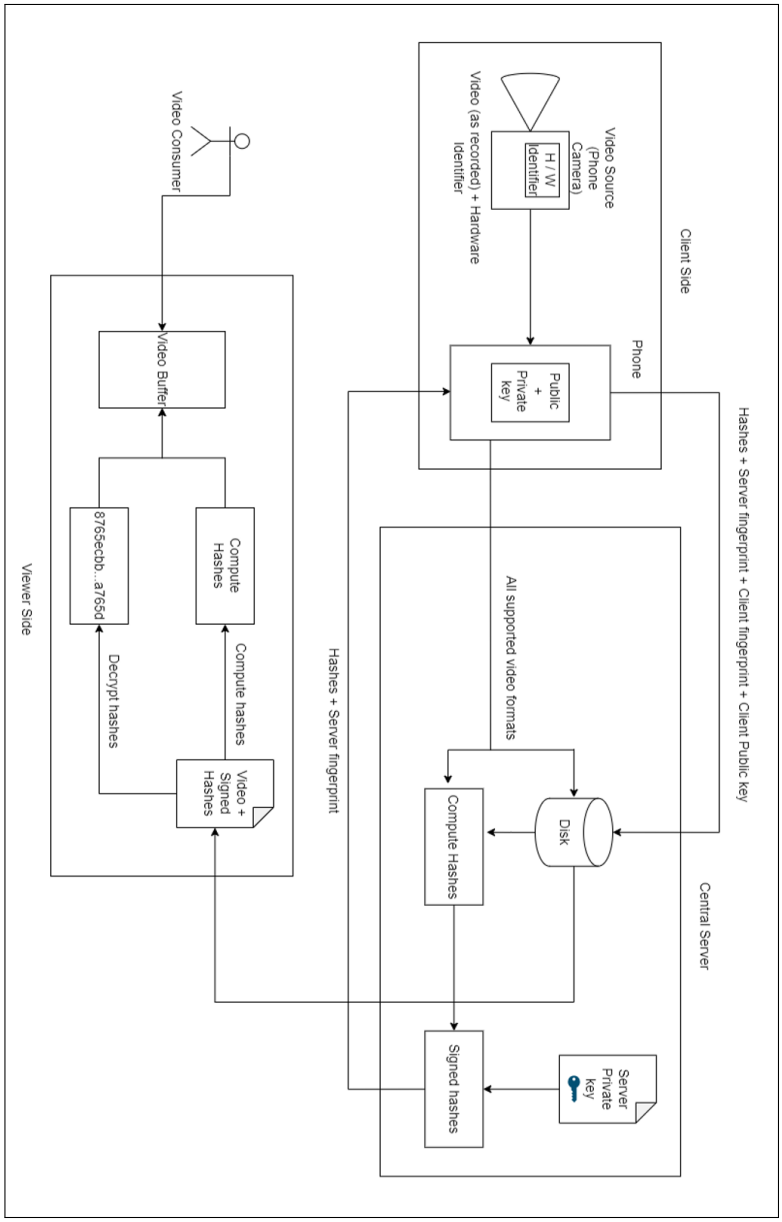

Figure 5.8: Data Flow

5.4 UML Diagrams

The Usecase diagram in Figure 5.4 depicts how content producers and consumers will interact using the system architecture for their own needs.

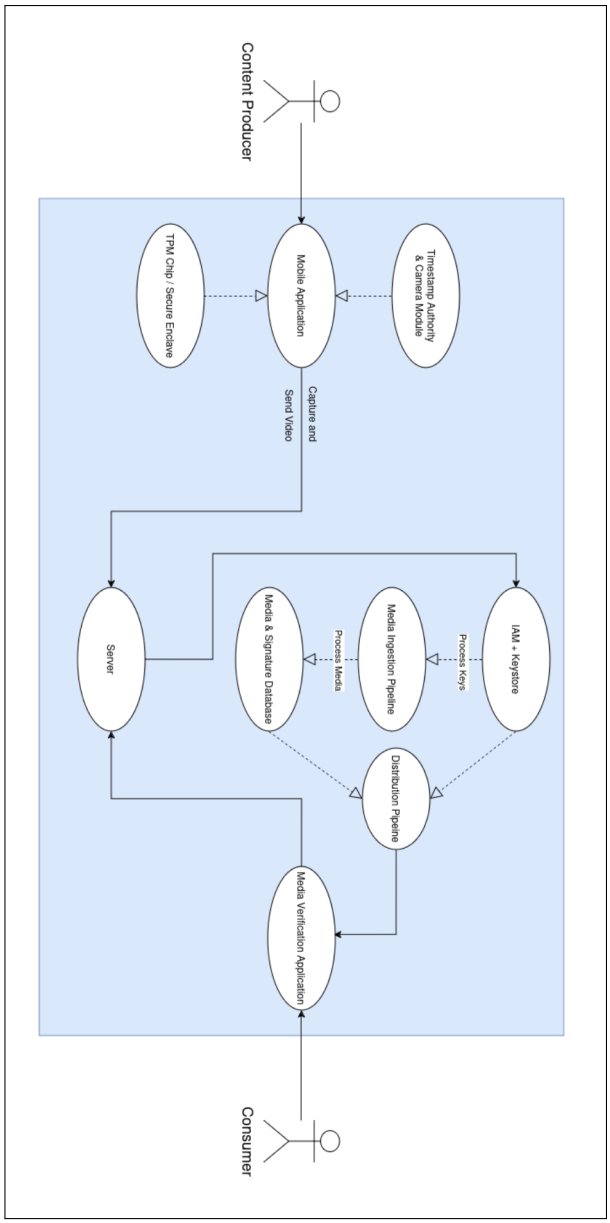

Figure 5.9: Use case diagram

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

I **Image Capture Module** - This module plays a crucial role in capturing and establishing authenticity of the image at the source level. It encompasses tasks such as capturing the image, calculating the hash of the image, signing the hash of the image using the unique private key of the device, packaging the image, signed hash and public key of the device, and communication with the server.

II **Image Steganography Module** - Offering a way to be able to transport images through a lossy medium and still retain provenance information, this module enables us to add provenance information directly at the pixel information of the image, which is resilient to compression and visually indistinguishable to the human eye. Through various encoding techniques such as QR code, Reed-Solomon Encoding and a unique Generative-Adversarial-Networks based approach, it retains provenance information inside the image even if the metadata is lost due to social media platforms.

III **Provenance Creation and Verification Module** - Using the Secure Enclave chips to provide unique hardware identifiers for every device that captures an image, this module is critical in linking an image to a particular hardware device. This helps us establish provenance directly at the hardware level, which remains tamper-proof throughout the entire process.

6.2 Tools and Technologies Used

- Python 3.10
- OpenCV
- SciPy
- PyWavelets
- QReader
- ZBar Library
- NumPy
- Pandas

- Django
- PostgreSQL
- Jupyter Notebook
- Google Colab
- Git & GitHub
- Azure VM
- Google Workspace

6.3 Device Selection

At the outset of our project, a critical decision involved selecting the target platform for implementing hardware-based image provenance. Our primary requirement was a robust, hardware-backed keystore for the secure generation and management of cryptographic keys directly on the device.

In our exploration of the Android ecosystem, we encountered a significant challenge: the diverse landscape of system-on-a-chip (SoC) manufacturers, including Qualcomm (Snapdragon), MediaTek (Dimensity), and Samsung (Exynos). This heterogeneity results in varied implementations of the hardware keystore and inconsistent APIs for accessing its functionalities. This fragmentation would have introduced considerable complexity in developing a unified and reliable provenance system across a wide range of Android devices.

Conversely, our investigation into the Apple ecosystem revealed a more cohesive and standardized approach. Apple employs its proprietary Secure Enclave, a dedicated hardware security module integrated into their devices. This Secure Enclave contains a unique, factory-provisioned root key that serves as the foundation for generating and protecting cryptographic keys. Crucially, the private keys managed within the Secure Enclave are inaccessible to the operating system and even to Apple itself. Instead, Apple provides a consistent set of high-level APIs that allow applications to perform cryptographic operations, such as signing and encryption, using these secure keys. This API consistency spans Apple's entire product line, including iPhones, iPads, Macs, and Apple Watches.

Given the unified hardware security architecture and consistent API access offered by Apple's Secure Enclave, we determined that it provided a significantly more feasible and reliable platform for developing our image

provenance system. Consequently, we made the strategic decision to focus our initial implementation efforts on iPhones as the primary media capture and hardware identification device.

6.4 Algorithm Details

6.4.1 QR Code Embedding using Wavelet DCT

The approach utilizes a combination of Discrete Wavelet Transform (DWT) and Discrete Cosine Transform (DCT). Data bits are embedded into selected mid-frequency DCT coefficients of blocks within specific DWT detail subbands of the image's luminance channel (Y). Robustness is enhanced via high error correction QR codes, data repetition, and careful coefficient selection.

6.4.1.1 Parameters and Constraints

- D_{in}: The input data string to be embedded.
- I_{host}: The original host image file.
- $\alpha \in \mathbb{R}^+$: Embedding strength parameter, controlling the magnitude of DCT coefficient modification.
- W: Wavelet type for DWT/IDWT (e.g., 'db4', 'bior4.4').
- $S_{embed} \in$ {'HL', 'LH', 'HH'}: Target DWT subband(s) for embedding.
- N_{dct}: DCT block size (typically $N_{dct} = 8$).
- C_{idx}: Set of selected 2D indices (r, c) within an $N_{dct} \times N_{dct}$ DCT block for embedding. $C_{idx} = \{(r_1, c_1), (r_2, c_2), \dots\}$. Lower-mid frequencies are preferred.
- $R \in \{3, 5, 7, \dots\}$: Odd integer repetition factor for redundant bit embedding.
- $E_{qr} \in$ {'L', 'M', 'Q', 'H'}: QR code error correction level (typically 'H' for highest robustness).

6.4.1.2 Key Components

- **Payload Data (P_{str}):** The information to be embedded.
- **Host Image (I_{host}):** The original image.
- **Stego Image (I_{stego}):** The image containing the embedded data.
- **Embedding Strength (α):** A scalar controlling the modification intensity of transform coefficients.
- **Wavelet Choice (W):** The mother wavelet for DWT (e.g., 'db4').
- **Target DWT Subband (S_{target}):** Typically a detail subband (HL, LH, HH) for embedding.
- **DCT Coefficients (C_{idx}):** Specific mid-frequency coefficients within DCT blocks targeted for modification.
- **Repetition Factor (R):** An optional odd integer for redundant bit embedding to enhance robustness.

6.4.1.3 Embedding Procedure

1. **Data Preparation:** The input payload P_{str} is encoded into a QR code. This process incorporates a chosen error correction level (preferably high). The binary representation of this QR code, along with metadata defining its dimensions, forms the bitstream $B_{payload}$ to be embedded. If $R > 1$, this bitstream is effectively repeated.

2. **Image Transformation:** The host image I_{host} is converted to the YCbCr color space. The luminance channel (Y) is selected for embedding due to its lower perceptual impact. A 2D DWT is applied to the Y channel, decomposing it into approximation (LL) and detail (LH, HL, HH) subbands. The designated S_{target} subband is chosen.

3. **Data Embedding:** The S_{target} subband is divided into non-overlapping blocks (typically 8×8). The 2D DCT is applied to each block. Specific mid-frequency DCT coefficients (defined by C_{idx}) within these blocks are then subtly modified to encode the bits from $B_{payload}$. The modification magnitude is governed by the embedding strength α. For a bit $b \in \{0, 1\}$, a coefficient v might be altered to $v' \approx v \pm \alpha_b$, where α_b depends on b.

4. **Image Reconstruction:** The modified DCT blocks are transformed back using an inverse DCT. These blocks form the modified S_{target} subband. An inverse DWT is then applied using the modified S_{target} and the original LL and other detail subbands to reconstruct the modified luminance channel Y'. Finally, Y' is combined with the original chrominance channels (Cb, Cr) and converted back to the RGB color space to produce the stego image I_{stego}.

6.4.1.4 Extraction Procedure

1. **Image Transformation:** The stego image I'_{stego} (potentially altered) undergoes the same initial transformations as in embedding: YCbCr conversion followed by DWT on the Y' channel to obtain the S'_{target} subband.

2. **Data Extraction:** The S'_{target} subband is processed block-wise. For each block, the DCT is computed. The values of the pre-defined DCT coefficients (from C_{idx}) are examined. The embedded bit is estimated based on the coefficient's value relative to its presumed original state or a decision threshold (e.g., its sign if α was dominant). This process reconstructs a raw bitstream $\hat{B}_{raw}$.

3. **Data Reconstruction:** If a repetition factor $R > 1$ was used, majority voting is applied to segments of $\hat{B}_{raw}$ to determine each bit of the payload metadata (QR dimensions) and the QR data itself. The reconstructed binary data is then used to form the QR code structure, which is subsequently decoded by a standard QR reader to retrieve the original payload $\hat{P}_{str}$.

6.4.2 Embedding Data using Reed-Solomon Encoding

This approach embeds data into the luminance channel (Y) of an image using the Discrete Cosine Transform (DCT). Error correction is applied to the payload and its length using Reed-Solomon (RS) codes. Embedding utilizes a quantization-based modification of a specific mid-frequency DCT coefficient within 8×8 blocks.

6.4.2.1 Key Components

- **Payload Data (P_{in}):** The byte sequence to be embedded.
- **Host Image (I_{host}):** The original image.

- **Stego Image (I_{stego}):** The image containing the embedded data.
- **Embedding Strength (S):** A scalar defining the quantization step size for DCT coefficient modification.
- **Target DCT Coefficient ((r_e, c_e)):** A specific mid-frequency coefficient index within each DCT block chosen for embedding.
- **Reed-Solomon ECC Symbols ($k_{len}, k_{payload}$):** Number of error correction symbols for payload length and payload data, respectively.

6.4.2.2 Embedding Procedure

1. **Data Preparation and Error Correction:** The length of the input payload P_{in} is first determined. Both this length information and the payload itself are independently encoded using Reed-Solomon codes, adding k_{len} and $k_{payload}$ error correction symbols respectively. These encoded segments are then concatenated and converted into a single binary string, B_{embed}, ready for embedding.
2. **Image Transformation:** The host image I_{host} is converted to the YCbCr color space. The luminance channel (Y) is selected for embedding. This channel may be padded to ensure its dimensions are multiples of the DCT block size (typically 8×8).
3. **Data Embedding in DCT Domain:** The (padded) luminance channel is processed in non-overlapping 8×8 blocks. For each block:
   - The 2D DCT is applied.
   - The value of the pre-selected DCT coefficient at index (r_e, c_e), denoted v, is retrieved.
   - If bits from B_{embed} remain, the current bit b is used to modify v to v' via a quantization rule. This rule ensures that $v' \pmod S$ falls into one of two distinct sub-intervals based on whether b is 0 or 1. For example, v' could be $\lfloor v/S \rfloor \times S + q_b \times S$, where q_b is a fraction like 0.25 for $b = 0$ and 0.75 for $b = 1$.
   - The modified coefficient v' replaces the original in the DCT block.
   - The inverse 2D DCT is applied to the modified block.

   This process continues until all bits in B_{embed} are embedded.

4. **Image Reconstruction:** The modified luminance channel is clipped to the valid pixel range and any padding is removed. It is then combined with the original chrominance channels (Cb, Cr) and converted back to the RGB color space to produce the stego image I_{stego}.

6.4.2.3 Extraction Procedure

1. **Image Transformation:** The stego image I'_{stego} (potentially altered) undergoes the same initial transformations: YCbCr conversion and padding of the Y' channel.

2. **Data Extraction from DCT Domain:** The (padded) Y' channel is processed block-wise. For each block, the DCT is computed. The value of the coefficient at (r_e, c_e), denoted $\hat{v}$, is extracted. The embedded bit $\hat{b}$ is estimated by determining which quantization sub-interval $\hat{v} \pmod S$ falls into (e.g., $\hat{b} = 1$ if $(\hat{v} \pmod S)/S > 0.5$, else $\hat{b} = 0$). All such estimated bits form the raw extracted bitstream $\hat{B}_{extracted}$.

3. **Data Reconstruction and Reed-Solomon Decoding:** The initial portion of $\hat{B}_{extracted}$ corresponding to the encoded length is converted to bytes and decoded using Reed-Solomon codes to retrieve the estimated payload length $\hat{L}$. Subsequently, the portion of $\hat{B}_{extracted}$ corresponding to the encoded payload is converted to bytes and decoded using Reed-Solomon codes (with $k_{payload}$ derived from $\hat{L}$) to recover the original payload $\hat{P}_{in}$. Error handling is applied during RS decoding.

6.4.3 Steganographic Watermarking using GAN-based Deep Learning approach

6.4.3.1 System Overview

Our system comprises three PyTorch models:

- **Embedder (aka. Generator)**
- **Extractor (aka. Decoder)**
- **Discriminator (aka. Critic/Detector)**

At training time, the embedder learns to imperceptibly hide a binary "payload" within a cover image; the extractor learns to recover it; and the discriminator learns to distinguish clean from watermarked images. Together

they form a GAN-like adversarial framework that balances fidelity (imperceptibility) against robustness (successful extraction).

At inference time, two Flask endpoints are provided:

1. `/encode`: Accepts an image and text, converts the text to a bit-tensor payload, runs the embedder, and returns the watermarked image.
2. `/decode`: Accepts an image, runs the extractor, thresholds and majority-votes the bits, and returns the hidden text.

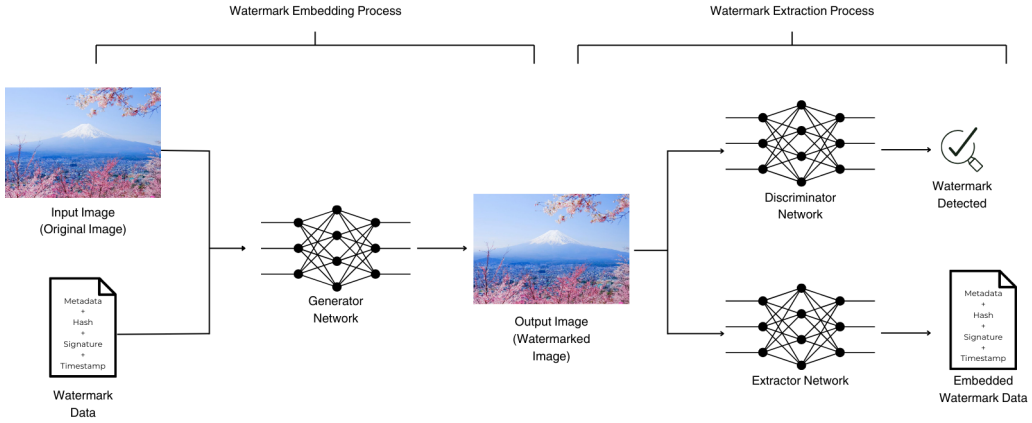

Figure 6.1: Overview of GAN-based watermarking models and process.

6.4.3.2 Data Preparation

- **Dataset**: PNG images of varying resolution, under `data/train` and `data/val`.
- **Loader**: Based on `torchvision.datasets.ImageFolder` with
  - Random horizontal flip
  - Random 360×360 crop
  - Normalize to mean=[.5,.5,.5], std=[.5,.5,.5]
- **Batch size**: 8; **Workers**: 8.

Images are normalized to $[-1, 1]$ during training and to $[0, 1]$ at inference.

6.4.3.3 Model Architectures

1. Embedder (Generator)

   - **Inputs**: Cover image tensor $(B, 3, H, W)$ in $[-1, 1]$, payload tensor $(B, 1, H, W)$ binary.
   - **Hidden size**: 32; **Data depth**: 1.
   - **Structure**: Four convolutional blocks:
     (a) Conv(3→32) + LeakyReLU + BatchNorm
     (b) Conv((32 + 1)→32) + LeakyReLU + BatchNorm
     (c) Conv((64 + 1)→32) + LeakyReLU + BatchNorm
     (d) Conv((96 + 1)→3)
   - **Skip connection**: Final output added to the original image.

2. Extractor (Decoder)

   - **Input**: Watermarked image $(B, 3, H, W)$.
   - **Structure**: Four conv layers:
     (a) Conv(3→32) + LeakyReLU + BatchNorm
     (b) Conv(32→32) + LeakyReLU + BatchNorm
     (c) Conv((64)→32) + LeakyReLU + BatchNorm
     (d) Conv((96)→1)
   - **Output**: $(B, 1, H, W)$ logits for payload bits.

3. Discriminator (Critic)

   - **Input**: Image $(B, 3, H, W)$.
   - **Intermediate depth**: 32.
   - **Layers**: Conv(3→32) → Conv(32→32) → Conv(32→32) → Conv(32→1), each with LeakyReLU + BatchNorm except the last.
   - **Score**: Global average of final feature map.

6.4.3.4 Training Process

Training runs for 35 epochs over train and validation sets:

1. **Critic update (Wasserstein-style)**:

- Loss = $\mathbb{E}[\text{critic(cover)}] - \mathbb{E}[\text{critic(generated)}]$.
- Backpropagate and clip critic weights to $[-0.1, 0.1]$.

2. **Generator & Decoder update**:
   (a) Generate watermarked = encoder(cover, payload).
   (b) Decode bits = decoder(watermarked).
   (c) Compute:

$$\begin{aligned} \text{Encoder MSE} &= \text{MSE}(\text{watermarked}, \text{cover}) \\ \text{Decoder BCE} &= \text{BCEWithLogits}(\text{decoded}, \text{payload}) \\ \text{Decoder acc} &= \frac{\#(\text{sign}(\text{decoded}) = \text{payload})}{\text{total bits}} \\ \text{Adv. score} &= \text{critic}(\text{watermarked}) \end{aligned}$$

   (d) Total loss: $100 \times \text{MSE} + \text{BCE} + \text{Adv. score}$.

3. **Validation**: Same forward steps, logging MSE, BCE, accuracy, critic scores, SSIM, PSNR, bits-per-pixel.

4. **Hyperparameters**:

- Learning rates: 1×10^{-4} for both Adam optimizers.
- Critic weight clipping: $[-0.1, 0.1]$.
- Loss trade-off: 100×MSE vs. BCE vs. adversarial.

6.4.3.5 Watermark Bit Pipeline

Encoding Text → Bits → Tensor

1. `text_to_bits`: Each character → 8-bit ASCII.
2. Append 000 as end marker after converting each character to it's 8-bit ASCII representation.
3. Tile message to fill $H \times W \times$ depth.
4. Reshape to $(1, 1, H, W)$ FloatTensor.

Embedding
Pass cover and payload into the embedder, clamp output to valid range, denormalize, and save.
Extraction

1. Normalize and run decoder to get logits.
2. Threshold logits to bits.
3. Group bits into bytes, split by previously used end marker value, convert back to text candidates.
4. Form and return the extracted watermark string by majority-voting character by character, using the corresponding characters from each candidate.

6.4.3.6 Deployment (Flask API)

- Models saved under `models_wmgan/`.
- Flask loads models in `.eval()` mode.
- `POST /encode`: multipart image + JSON text → watermarked image.
- `GET /decode`: image upload → extracted string.

CHAPTER 7
SOFTWARE TESTING

7.1 Test Environment and Tools

- Software Stack
  - **Mobile app (Creator):** iOS Simulator (iOS 14+) running Swift/CryptoKit
  - **Server (Verifier):** Azure Virtual Machine, running Django server
- Test Data
  - **Images:** Seven base images of varying characteristics for testing steganographic watermarking methods.
    1. Small (256 × 256 px, color)
    2. Medium (512 × 512 px, color)
    3. Medium-Large (1024 × 768 px, color)
    4. Large (1920 × 1080 px, color)
    5. Extra-Large (4096 × 2160 px, color)
    6. Transparent background (1024 × 1024 px, PNG)
    7. Black white (1024 × 1024 px, grayscale)
  - **Payload:** Four watermark strings embedded per image.
    1. Small (32 char ASCII)
    2. Medium (128 char ASCII)
    3. Large (512 char ASCII)
    4. Special-character small (32 char with symbols)

7.2 Functional Verification

1. **Metadata Embedding and Extraction**
   - **Procedure**
     - Capture media via the mobile app.
     - Embed trusted timestamp, device signature (TPM-derived), and SHA-256 hash into EXIF/XMP metadata.
     - Upload to server, extract metadata.
   - **Success Criteria:** All fields match original values with zero bit errors.
   - **Results:** 100% extraction accuracy across all test files.

2. **Hash Recalculation & Signature Validation**

   - **Procedure**
     - Compute SHA-256 of uploaded media.
     - Compare to embedded hash; validate TPM signature via public key.
   - **Success Criteria:** Pass for untampered files; fail for any bit-level modification.
   - **Results:** Untampered: all passed

7.3 Watermark Robustness Tests

7.3.1 Evaluation Metrics

1. **Fidelity**

   - Peak Signal-to-Noise Ratio (PSNR, dB).
   - Structural Similarity Index (SSIM, 0–1)

2. **Robustness**

   - Bit Error Rate (BER) of extracted payload vs. original.
   - Watermark detection under attacks: JPEG compression (Q=50, 70, 90), Gaussian noise (=5, 10), random cropping (up to 25% area), rotation (±5°), scaling (0.5×–2×).

CHAPTER 8
RESULTS

8.1 Result

The experimental evaluation confirms that Project MAVIS reliably embeds and verifies both metadata and steganographic watermarks, offering high integrity assurance even under adversarial conditions. Below is a summary of key findings across fidelity, robustness of the 3 steganographic watermarking techniques.

8.2 Fidelity (PSNR SSIM)

1. **GAN-based** watermarking preserved highest perceptual quality (PSNR 40 dB, SSIM 0.99).
2. **QR-DCT** delivered acceptable quality (PSNR ˜39 dB), but exhibited block artifacts at large payloads.
3. **Reed–Solomon** variant traded off more fidelity (PSNR 36 dB) for improved error correction.

8.3 Robustness (BER under Attacks)

1. **Reed–Solomon** enhanced QR watermarking shows the lowest BER across all distortions, demonstrating strong error-correction.
2. **GAN-based** approach is relatively robust to mild compression/noise but degrades under geometric transforms due to random cropping during training.
3. **QR-DCT** is most vulnerable, particularly to cropping and reed solomon, GAN based approach were vulnerable to both cropping and rotation. GAN-based is computationally heaviest but still completes in ¡ 100 ms server-side.

These results validate that Project MAVIS meets its objectives: providing a tamper-proof, scalable, and resilient media authenticity system suitable for use cases in insurance, legal evidence, journalism, and surveillance.

CHAPTER 9
OTHER SPECIFICATIONS

9.1 Advantages

This project offers significant advantages by providing a proactive solution to media authenticity, leveraging cryptographic techniques to ensure tamper-evident media verification. It enhances trust in digital content by securing media at its source, preventing the spread of misinformation and deepfakes. The system also supports secure cross-platform media sharing while maintaining the integrity and provenance of the content, making it highly valuable in fields like journalism, law enforcement, and document sharing, etc.

9.2 Limitations

There are a few limitations to this project.

1. The platform is currently restricted to devices with a TPM or TEE such as iOS devices with their secure enclave technology, limiting its accessibility to users on other platforms, such as Android.

2. The project only supports Images, meaning that other types of media like videos and audio are not included, which restricts its broader use in multimedia contexts.

3. Dependent on a central server for verifying Image signatures (in cases of where Metadata is of the image is lost), which could pose challenges related to server availability and security.

4. Does not address legal enforcement or the regulatory challenges associated with manipulated content.

CHAPTER 10
CONCLUSION & FUTURE WORK

10.1 Conclusion

In conclusion, our research on content provenance has given us a solid understanding of the current state of digital media verification and tamper-resistant technologies. By reviewing key papers and existing solutions, we have gained insights that inform the direction of our project. We have successfully built a proof-of-concept that enables users to securely capture and distribute images, while anyone with an image can verify with a high degree of trust, that the image they have is authentic or not. Our project uses the Secure Enclave within Apple devices to allow images to be traced back to the exact hardware of the device that captured it, while also employing techniques like digital signatures, QR Code embedding using DCT Wavelets, Reed-Solomon Encoding and a unique Generative-Adversarial Networks based approach to embed data inside the pixel data of the image, allowing for a high degree of trust in the image. Given the rapid rise of generative AI and its potential for misuse, we believe that our proposed solution holds significant promise.

10.2 Future Work

1. **Expanding Media Support**: Incorporate additional media types and formats beyond images, including audio, video, and documents.
2. **SNARK + STARK Integration**: Implement Succinct Non-Interactive Arguments of Knowledge (SNARKs) and Scalable Transparent Arguments of Knowledge (STARKs) for establishing a robust chain of trust across media interactions.
3. **Developing a Custom Codec**: Design and implement a proprietary codec optimized for embedding tamper-evident signatures and secure metadata in media files.
4. **Cross-Platform Integration**: Ensure compatibility and seamless integration across various platforms and ecosystems, allowing media to be verified across multiple devices and services.
5. **Hardware-Level Codec Integration**: Integrate the custom codec at the chip level within electronic devices such as smartphones and cameras, enhancing media security from the point of capture.

CHAPTER 11
APPENDIX

11.1 Appendix A

- **Technical Feasibility:**
  - Strong Foundation: Apple's Secure Enclave offers robust, hardware-backed security for key management and cryptography.
  - Simplified Development: Consistent and well-documented high-level APIs streamline integration.
  - Reduced Complexity: Homogenous iOS ecosystem minimizes cross-device compatibility issues.
  - Established Cryptography: Utilizes well-known techniques for signing and verification.
  - Potential Challenges: Performance optimization and seamless user integration for verification.
- **Economic Feasibility:**
  - Development Costs: Significant initial investment in software engineering and testing.
  - Cost Mitigation: Focus on iOS reduces fragmentation compared to Android.
  - Market Demand: Dependent on the need for verifiable image provenance (e.g., journalism, legal, IP).
  - Monetization Strategies: Potential for one-time purchase, subscriptions, or platform integration.
  - Key to Viability: Balancing investment with revenue and perceived value of trust.
- **Operational Feasibility:**
  - Manageable Core Functionality: Relies on the device's Secure Enclave, minimizing external infrastructure for initial stages.
  - Seamless User Interaction: Potential for integration within the camera app or as a post-capture option.
  - Adoption Challenge: Educating users on the benefits of image provenance is crucial.
  - Verification Mechanism: Requires a clear and accessible way for third parties to verify provenance.

- Scalability: Achievable through cloud infrastructure for any off-device components.
- Key to Success: Intuitive design and effective third-party verification.

11.2 Appendix B

11.2.1 Survey Paper Details

Paper Status: Presented at conference, applied for publication.
Date of Conference: 29th March, 2025
Name of Conference: International Conference on Computing Systems and Intelligent Applications (ComSIA 2025)

11.2.2 Implementation Paper Details

Paper Status: Applied
Name of Conference: 16th ICCCNT (INTERNATIONAL IEEE CONFERENCE ON COMPUTING, COMMUNICATION AND NETWORKING TECHNOLOGIES)
Conference Webpage: https://16icccnt.com/index.php

11.3 Appendix C

11.3.1 Plagiarism Report

The plagiarism report for the project report has been attached herewith.
Similarity: 6%

Figure 11.1: Plagiarism Report from Urkund

Figure 11.2: InC Certificates

StellarBlocks LLC

7345 164th Ave NE
STE i145-1497
Redmond WA 98052 USA

Sponsorship Letter

October 1st 2024

This is to inform whomever it may concern that StellarBlocks LLC, WA, USA, will support the BEProject for the following students, and I will act as their external mentor/guide for the period September 2024 to June 2025.

1. Chinmay Patil
2. Omkar Wagholikar
3. Shantanu Wable
4. Soaham Pimparkar

The title of the project is "Project MAVIS: Media Authenticity Verification and Integrity System" ("Project").

The work done and the intellectual property created as part of this project is owned by StellarBlocks LLC as per the "Invention Assignment and Confidentiality Agreement" signed between the above students and StellarBlocks LLC. StellarBlocks LLC may provide support, at its discretion, for procuring software, subscriptions, licenses, or any other resources required for completion of the Project until June 30th 2025.

A letter of completion will be provided upon completion of the Project in May 2025.

Tushar Sugandhi
Founder & CEO

StellarBlocks LLC

Figure 11.3: Sponsorship Certificate

ComSIA-2025

Springer

acadroid

2nd INTERNATIONAL CONFERENCE ON COMPUTING SYSTEMS AND INTELLIGENT APPLICATIONS

Certificate of Participation

This is to certify that **Prof./Dr./Mr./Mrs. Soaham Pimparkar** is a presenter/co-author of the paper titled **State of Media Provenance: A Brief Survey** presented at the **2nd International Conference on Computing Systems and Intelligent Applications (ComSIA- 2025)** organized jointly by Campus of Open Learning, University of Delhi, and Shaheed Rajguru College of Applied Sciences for Women, University of Delhi, on **28th-29th March 2025.**

Prof. Payal Mago
Patron, Director
Campus of Open Learning,
University of Delhi, Delhi

Prof. Aboul Ella Hassanien
General Chair
Kuwait University, Kuwait

Created using pdfMachine merge by Broadgun Software

Figure 11.4: Survey Paper Certificate

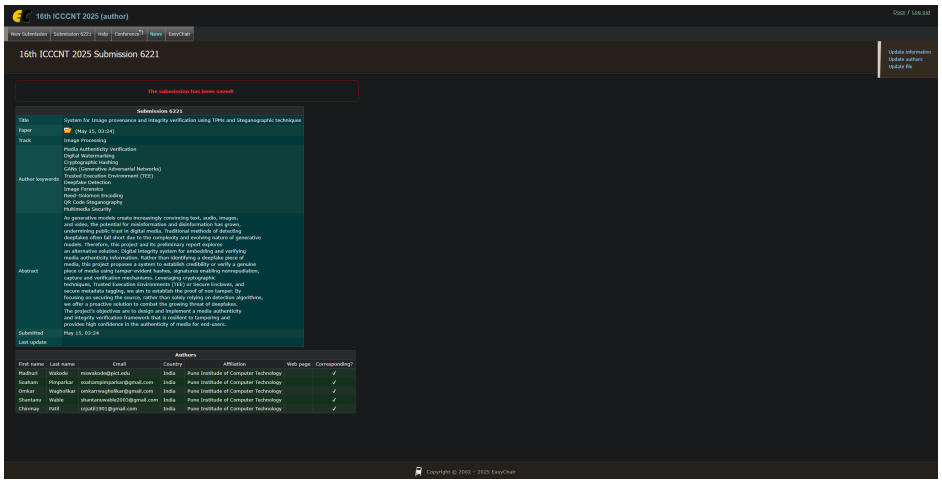

Figure 11.5: Research Paper Submission

CHAPTER 12
REFERENCES

[1] Feng, KJ Kevin, et al. "Examining the Impact of Provenance-Enabled Media on Trust and Accuracy Perceptions." Proceedings of the ACM on Human-Computer Interaction 7.CSCW2 (2023): 1-42.

[2] Bhowmik, Deepayan, et al. "An International Standard For Assessing Trustworthiness In Media." 2024 IEEE International Conference on Image Processing (ICIP). IEEE, 2024.

[3] The Content Authenticity Initiative (2020). Setting the Standard for Digital Content Attribution. https://c2pa.org/about/resources/

[4] Black, Alexander, et al. "Vpn: Video provenance network for robust content attribution." Proceedings of the 18th ACM SIGGRAPH European Conference on Visual Media Production. 2021.

[5] Andriushchenko, Maksym, et al. "ARIA: Adversarially Robust Image Attribution for Content Provenance." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022.

[6] Zhang, Jialong, et al. "Protecting intellectual property of deep neural networks with watermarking." Proceedings of the 2018 on Asia conference on computer and communications security. 2018.

[7] Van Le, Tri, and Yvo Desmedt. "Cryptanalysis of UCLA watermarking schemes for intellectual property protection." Information Hiding: 5th International Workshop, IH 2002 Noordwijkerhout, The Netherlands, October 7-9, 2002 Revised Papers 5. Springer Berlin Heidelberg, 2003.

[8] Petrangeli, Stefano, et al. "Integrating Content Authenticity with DASH Video Streaming." Proceedings of the 15th ACM Multimedia Systems Conference. 2024.

[9] Sedlmeir, Johannes, et al. "Battling disinformation with cryptography." Nature Machine Intelligence 5.10 (2023): 1056-1057.

[10] Fotos, Nikolaos, and Jaime Delgado. "Towards Privacy-Enhancing Provenance Annotations for Images." 2024 IEEE International Conference on Image Processing (ICIP). IEEE, 2024.

[11] Simmons, John C., and Joseph M. Winograd. "Interoperable Provenance Authentication of Broadcast Media using Open Standards-based Metadata, Watermarking and Cryptography." arXiv preprint arXiv:2405.12336 (2024).

[12] Asnani, Vishal, et al. "ProMark: Proactive Diffusion Watermarking for Causal Attribution." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024.

[13] Bureacă, Emil, and Iulian Aciobăniței. "A Blockchain Blockchain-based Framework for Content Provenance and Authenticity." 2024 16th International Conference on Electronics, Computers and Artificial Intelligence (ECAI). IEEE, 2024.

LIST OF ABBREVIATIONS

Abbreviation	**Illustration**
C2PA	Coalition for Content Provenance and Authority
TPM	Trusted Platform Module
SE	Secure Enclave